

P A P E R R E V I E W

REASONING

BANK

ReAct

REFLEXION

2025. 10. 29

<ReAct 2023/03/10>

ReAct: Synergizing Reasoning and Acting in Language Models - Shunyu Yao et al.

<Reflexion 2023/10/10>

Reflexion: Language Agents with Verbal Reinforcement Learning - Noah Shinn et al.

<Reasoning Bank 2025/09/29>

ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory - Siru Ouyang et al.

1 LLM limitations

2 ReAct

3 REFLEXION

4 REASONING BANK

5 Code Implement

Process

1) Thought

모델이 답을 내기 전에, 다음 단계/행동을 정하기 위해 언어로 적는 중간 추론 및 계획 과정

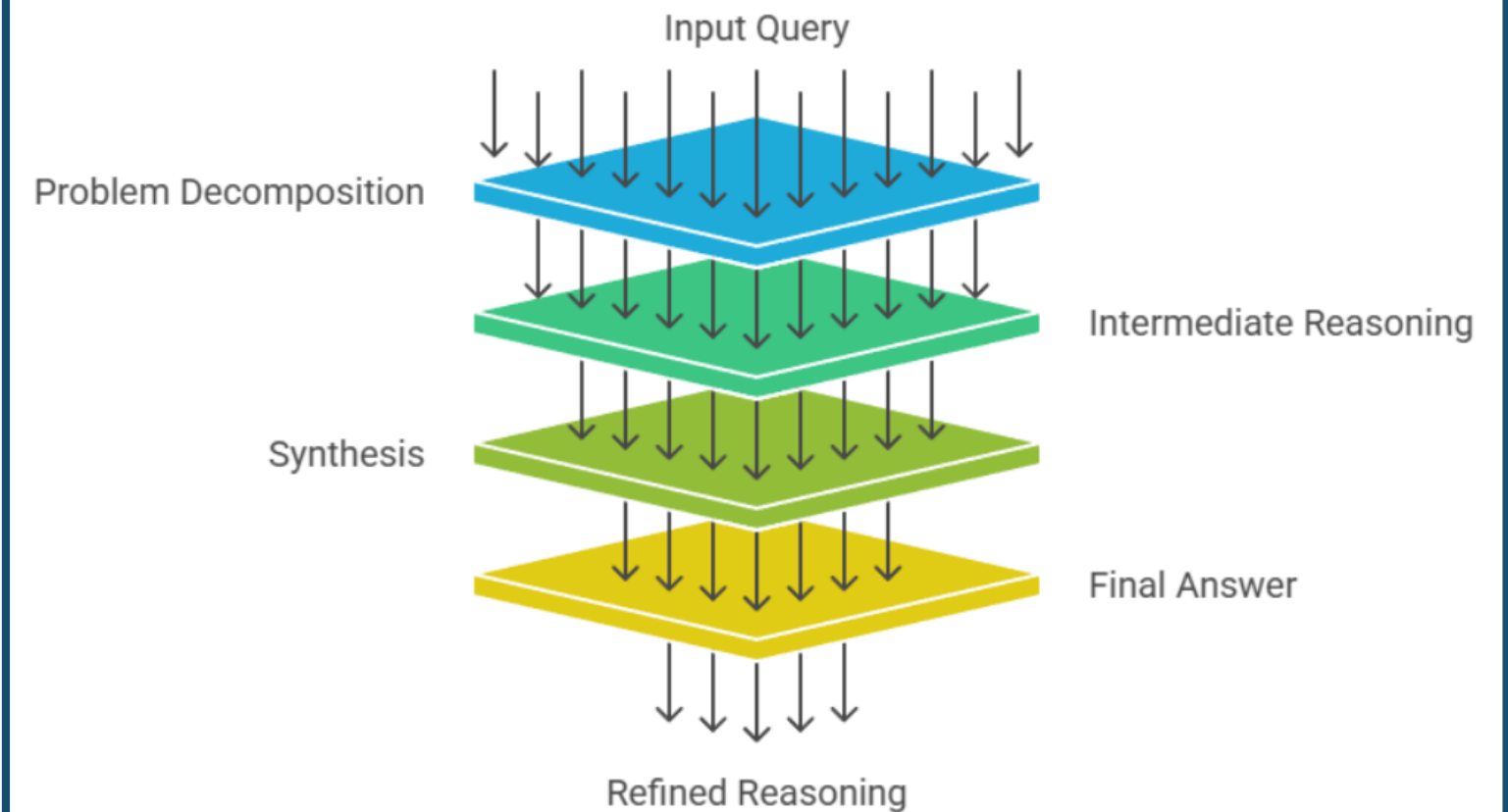
2) CoT (Chain of Thought)

연속된 중간 추론을 Thought의 텍스트로 풀어 더 정확한 답변을 생성하는 과정

3) Action

LLM 외부의 사전에 정해진 Tool을 사용하여 Thought의 결과를 검증하여 더 정확한 답변을 생성할 수 있는 프롬프팅 과정

Chain of Thought Reasoning Process



Halucination

1) 원리

- LLM 모델은 '모름' 이라는 답변을 제공하기보다 가장 그럴듯한 다음 토큰을 예측하는 성질을 가졌기 때문에 나타나는 현상

2) 영향

- 사실 오류 (숫자, 날짜, 인명 등 사실 틀림)
- 추론 오류 (계산, 논리가 어긋난 결론)
- 출처 부재 (근거를 거짓 생성 ex. 가짜 논문)
- 지시 불이행 (요구 형식을 무시)

Non-Determinism

1) 원리

- 모델의 확률적 토큰 생성과 생성 과정에서의 임의성으로 인해 같은 입력/ 같은 설정임에도 출력이 달라지는 현상

2) 영향

- 재현성 저하 (실험 비교 불가)
- 평가 편향 (한 번 잘 나온 답변을 정답으로 오판)
- 품질/신뢰성 관리 어려움
- 감사 불가능 (답변에 대한 설명 난이도 증가)

Reasoning + Action

1) Thought - Action - Observation

Thought(생각) - Action(행동) - Observation (결과)를 갱신하는 프롬프트 패턴과 에이전트 루프

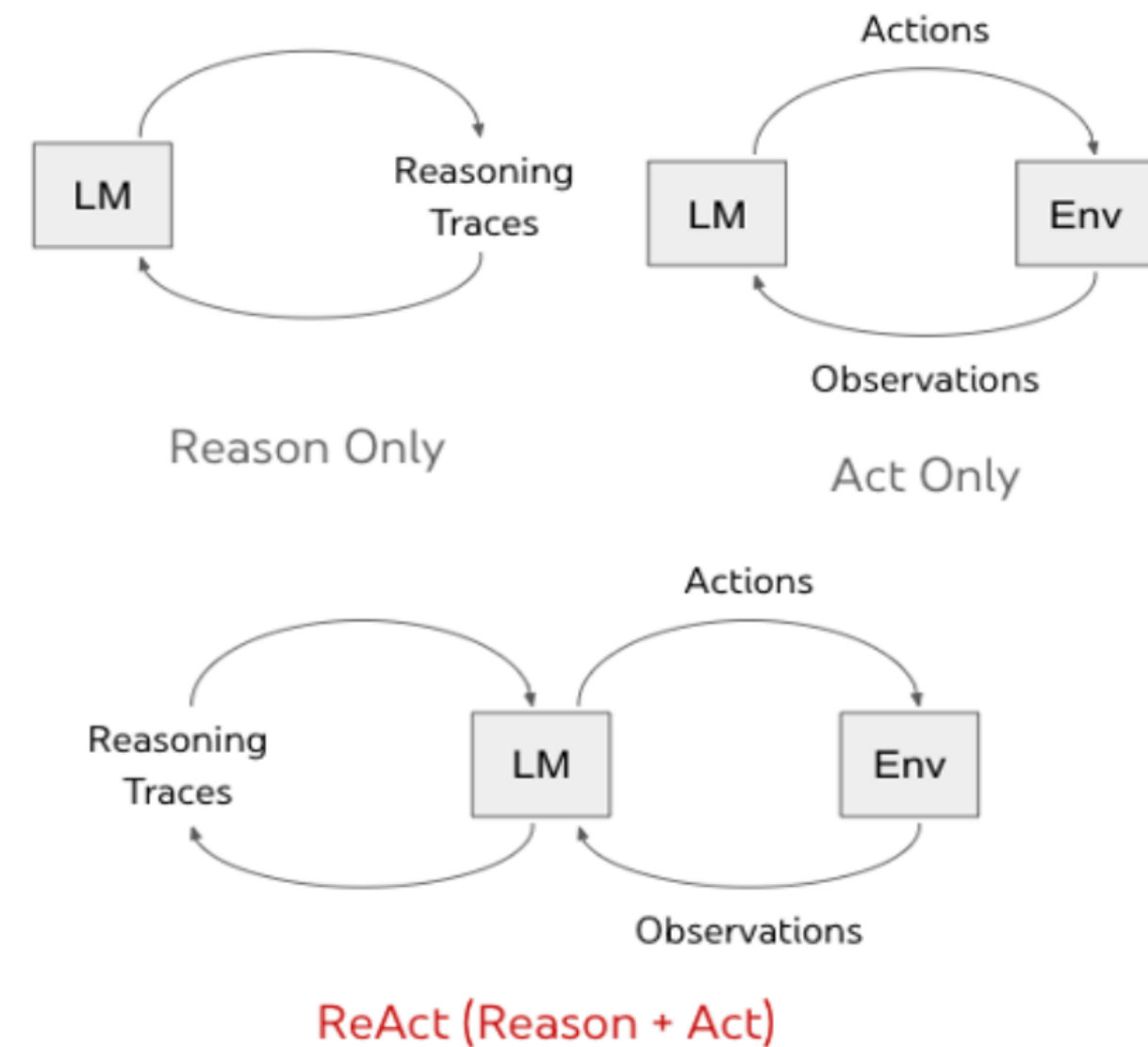
2) 설명

Thought -> Pred1를 추론(내부 일관성 추론)

Action -> Tool을 사용하여 Pred1를 검증하는
Pred2를 추론(외부 검증 추론)

Observation -> Pred2를 외부에서 검증해서 돌아오는 피드백

Thought - Action - Observation - (반복) - 결과



Components

1) Thought

쿼리/프롬프트를 받아서으로 답변을 생성

입력: (최초) user prompt /

(n회차) observation 결과

출력: (반복) 새로운 prompt 명령 / (최종) 답변

2) Action

Thought 의 출력을 검증하기 위해 툴을 호출하고 어떤 방식으로 평가할지 제시

입력: Thought의 출력

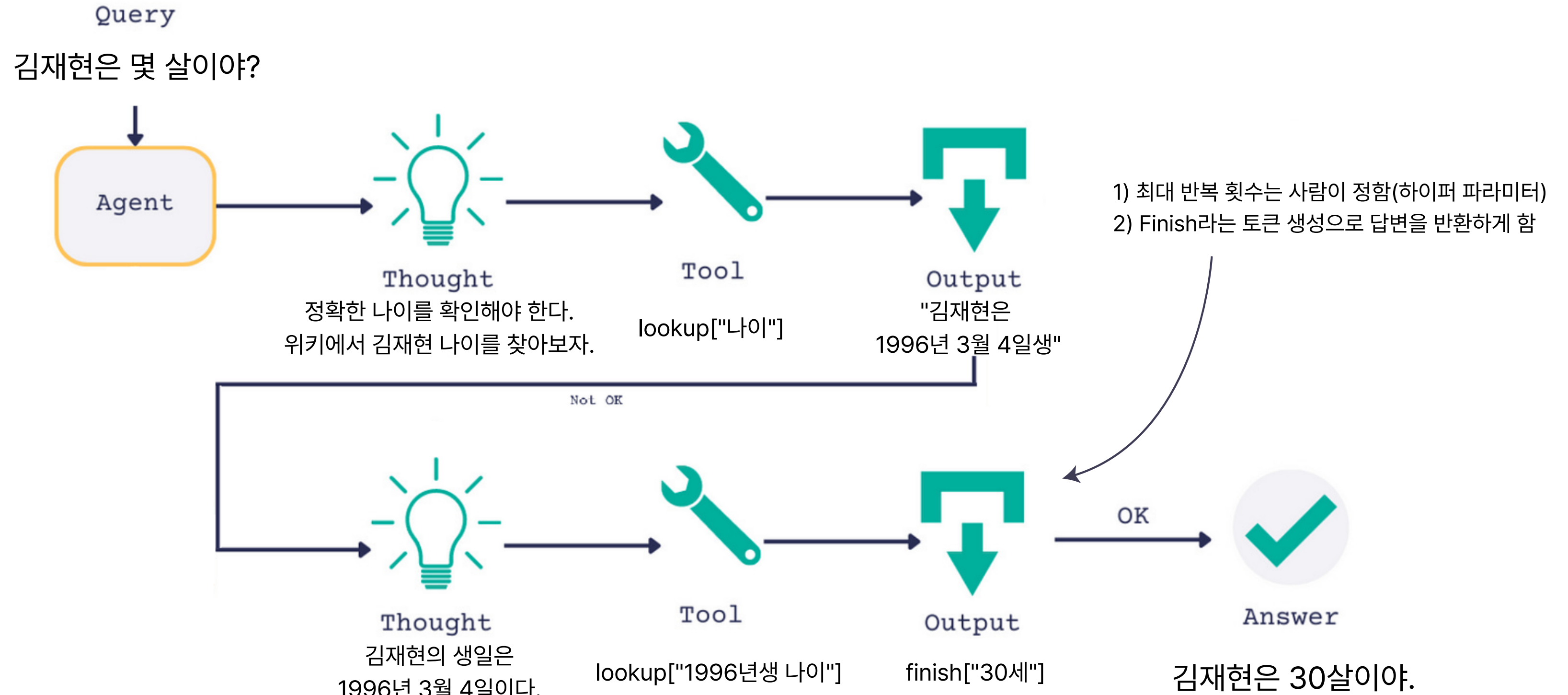
출력: Tool + 호출 명령어

3) Observation

Action에서 호출한 툴로 출력된 평가 결과의 핵심을 요약하여 보관

입력: Action Tool + 호출 명령어

출력: (n회차) Thought 출력의 검증 결과 /
(최종) 'finish'+검증 결과



Verbal Reinforcement Learning

1) 말로 하는 강화학습

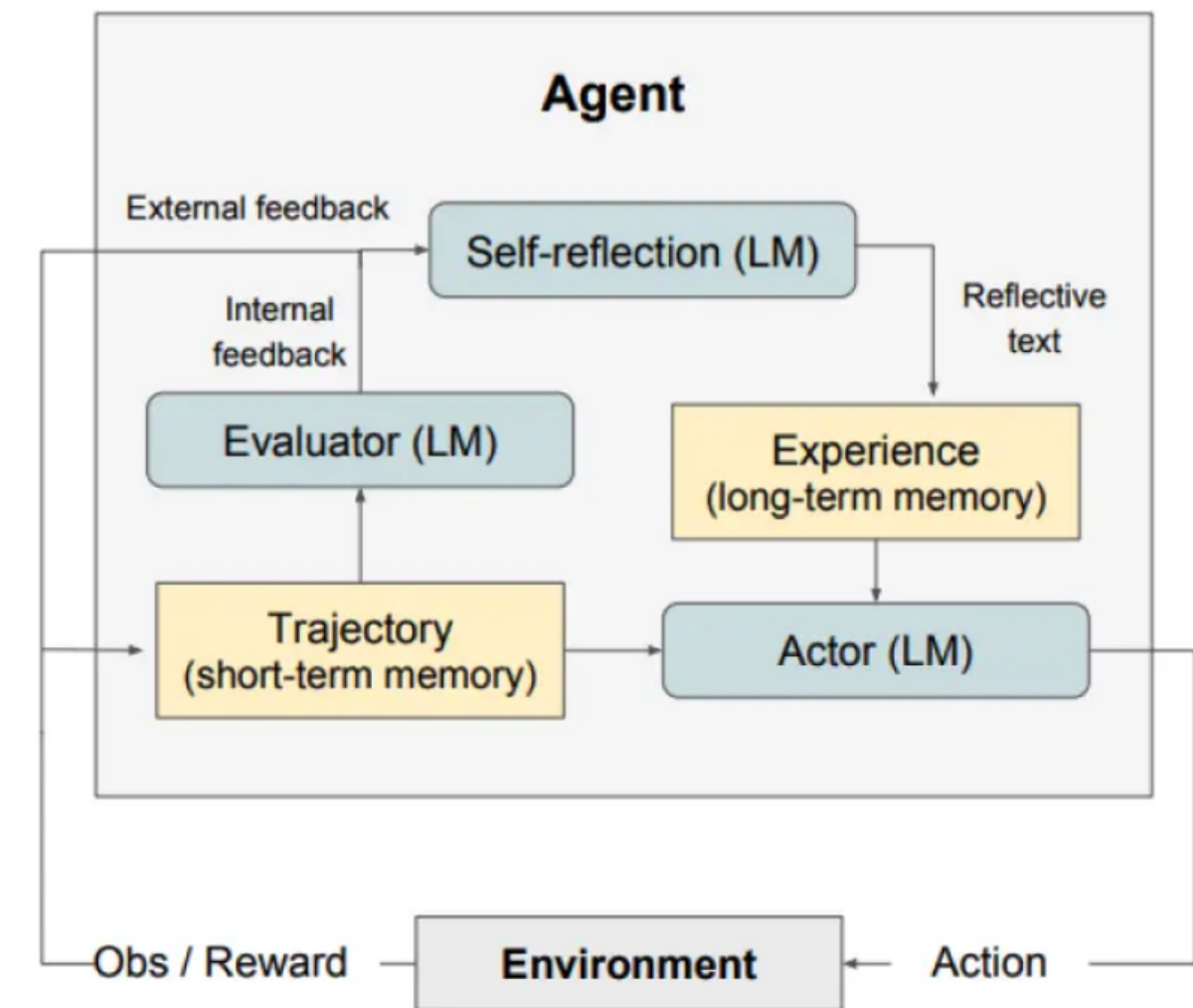
input에 대한 output 1을 평가하여 메모리에 저장하고 다음 output 2를 만들 때 메모리의 output 1의 평가 결과를 참조하여 더욱 좋은 결과를 반환하도록 하는 강화학습

2) Eval Tools

Internal 평가: 형식 준수, 간단 산술, 논리 일관성

External 평가: 사실성/최신성, 데이터 의존 과제

3) Task(in) - Trajectory - Eval. - Reflexion - next
Trajectory - Eval. - Reflexion -...- finish(out)



Components

1) Actor

실제 문제를 푸는 LLM으로 ReAct/CoT와 같은 에피소드를 생성하는 모델

입력: (최초) 프롬프트 / (n회차) 프롬프트 + mem

출력: (n회차) Thought, Action / (최종) 답

2) Evaluator

지금 생성된 답변을 평가해 성공/실패/점수/근거를 판정하는 내/외부 톨

입력: Actor에서 생성된 Action

출력: 긍정/부정적 평가

긍정(Pass/Support): Early_Stop / Best 저장 / 성공을 메모리에 저장하여 유사 Task에 활용 / 로그 고정
-> 사용된 Observation, 톨, Step, 신뢰도 등을 기록해 테스트 기준으로 사용함

부정(Fail/Refutes): 자기 반성 트리거 생성 후 메모리에 저장

-> Self-Reflexion을 통해

[Fail Reason + Avoid + Redirect + Trigger]

의 형태로 memory에 저장되어 다음 에피소드에서 해당 내용을 프롬프트 맨 앞에 주입

-> 최대 반복횟수에 도달했는데 여전히 부정 평가를 받고 있으면, 이전에 가장 좋은 답변을 반환하거나 불확실/거부 로 마무리 하는 것이 일반적

Components

3) Self-Reflection

에피소드 결과(Action)와 평가 결과를 자연어 교훈으로 요약하여 다음 에피소드에서 정답을 찾는 방식
는 정책을 바꿀 힌트를 제공

입력: Action + Evaluation 결과

출력: 자연어 교훈

교훈 구성)

Fail Reason: 실패 유형

ex. 빈 검색 루프, 잘못된 툴, 근거 누락 등

Avoid: 참조하지 말아야하는 목록을 제공

ex. "이 경로/쿼리/툴은 사용하지 말 것"

Redirection: 대안 제시, 동의어 추가 등 방법 제시

ex. "대신 @@ 툴을 사용하라."

Trigger: 교훈이 제공되는 조건을 제시

ex. "빈 검색 쿼리가 2회 이상일 때"

4) Memory

Self-Reflection에서 만든 교훈을 저장하고, 다음 에피소드 시작 시 프롬프트 앞부분에 주입해 Actor의 행동 선택을 유도

입출력: 자연어 교훈

에피소드 메모리: 같은 Task를 재시도할 때 사용, 최근 N개를 저장하여 다음 에피소드에 사용하도록 Reflexion 과정에서 주입

일반화 메모리: 비슷한 유형의 Task를 처리할 때 재사용 가능한 규칙을 저장하여 속도와 정확도 강화

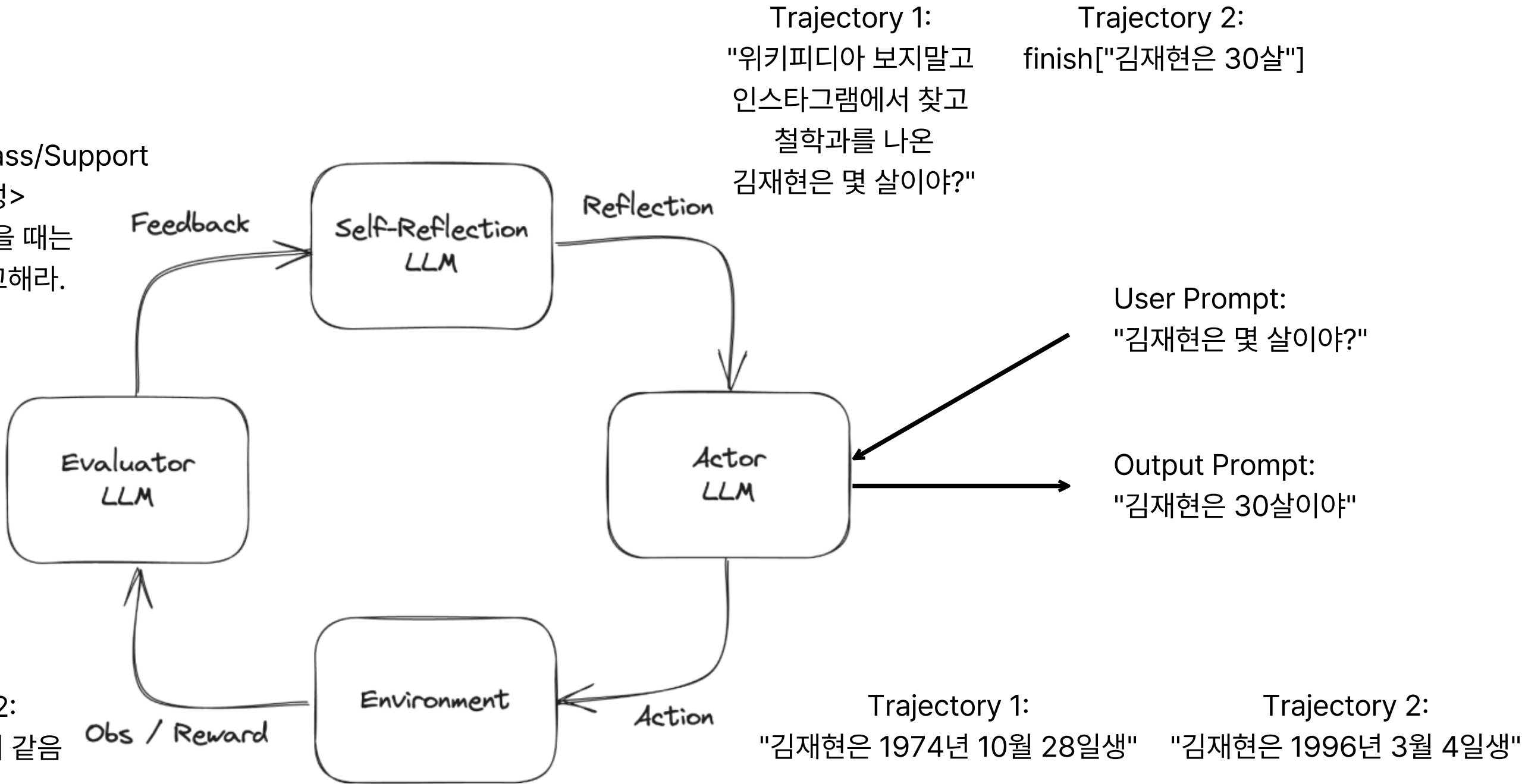
일반적인 사용법: 에피소드 메모리와 일반화 메모리의 교훈이 충돌할 경우 최근 교훈 우선, 교훈은 주기적으로 비워서 메모리 용량 및 속도 관리, 적당한 길이의 자연어로 변환하여 길이를 관리함

Trajectory 1: Fail/Refutes
<자연어 교훈 생성>
Fail reason: 잘못된 툴
Avoid: 위키피디아 보지마.
Redirection:인스타그램 찾아봐.
철학과 나온 사람 찾아봐.
Trigger: no_answer>2

Trajectory 2: Pass/Support
<자연어 교훈 생성>
이제 김재현을 찾을 때는
인스타그램을 참고해라.

Trajectory 1:
그 김재현은 야구선수 김재현임.
그 사람 아님

Trajectory 2:
그 김재현 맞는 거 같음
그 사람 아님



Reasoning Bank

1) 이름 비유의 의미

과거 Task에서 뽑은 추론 전략을 예금처럼 넣어 두었다가 다음 Task에서 꺼내서 사용하는 은행 같은 전략 저장소

2) 무엇을 예금하나?

전이 가능한 추론 패턴 전략을 요약해 메모리 아이템에 저장

3) 어떻게 인출하나?

새 Task가 오면 새로운 Task의 컨텍스트와 유사한 아이템을 검색해 시스템 지시문에 주입

과거 Task의 추론 전략 → 새로운 Task 제시 → 과거 추론 전략 꺼내기

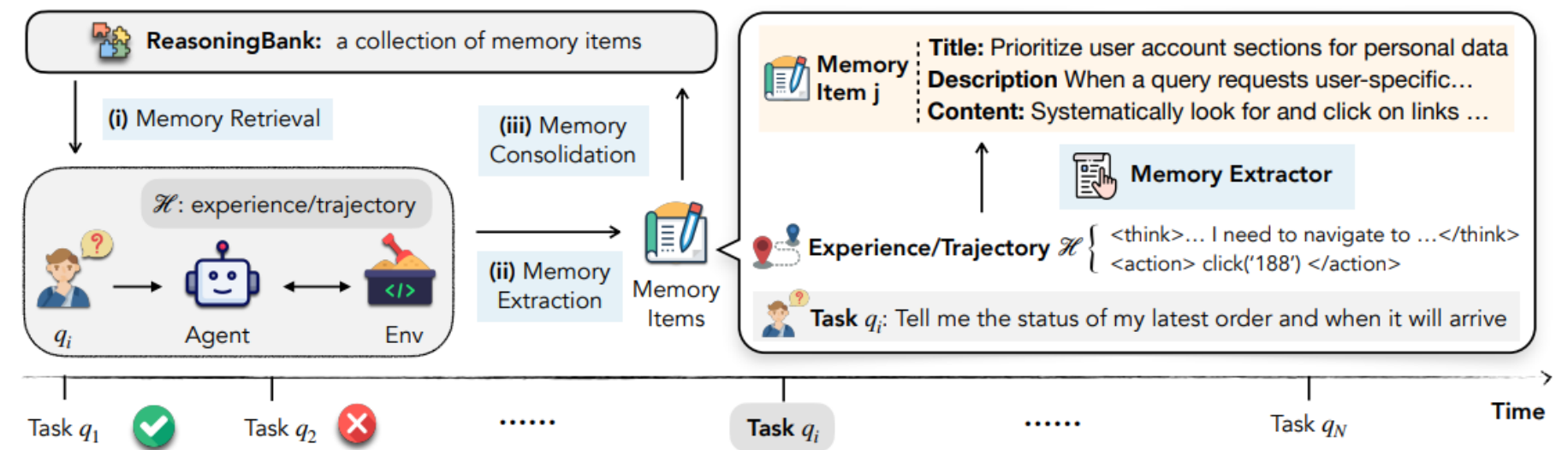


Transferable Reasoning Strategy

1) 전이 가능한 추론 전략
에이전트가 과거 Task의 에피소드(성공 or 실패)에서 전이 가능한 추론 전략을 뽑아서 구조화하여 메모리에 저장하고 다른 도메인(데이터 or 요청 or Task)에서도 그 전략을 사용하게 함

2) 자기 강화
다른 도메인에서 재사용된 그 전략이 여러 Task를 거치면서 자기 강화하게 됨

오랜 시간/ 여러 회차 재사용되면 풍부한 경험으로 더 좋은 메모리 -> 더 좋은 성과로 반복됨



Components

1) Retrieval

현재 질의(Task)상황에 유한 전략 메모리(top-k개)를 Reasoning Bank에서 임베딩 기반 유사도 검색으로 꺼냄

꺼낸 전략은 프롬프트 상단에 주입

이유: 불필요한 탐색을 줄이고 과거 오류를 반복하지 않기 위함(re-rank, 항목당 2~5줄 요약으로 넣음)

2) Construction

한 번의 에피소드가 끝날 때, 결과를 평가하고 그 신호를 바탕으로 새로운 전략 메모리를 여러 개 생성
성공한 결과는 검증된 전략으로 쓰이고, 실패한 결과는 피해야하는 신호로 쓰임

이유: 원시 궤적은 길고 곁끄러움, 그래서 인간이 읽기 쉽고, 기계가 쓰기 쉬운 구조화된 메모리로 정리해야 재사용이 용이함

3) Consolidation

Construction으로 만들어진 새로운 전략 메모리를 Reasoning Bank에 추가해 점진적/누적적 레포지토리로 유지

이유: 페루프(단힌 루프)를 만들기 위한 과정

과거 경험 활용 -> 새 메모리 추출 -> Bank

Memory 갱신 -> 재사용

위 루프를 반복해 지속적으로 강화/진화 하게 됨

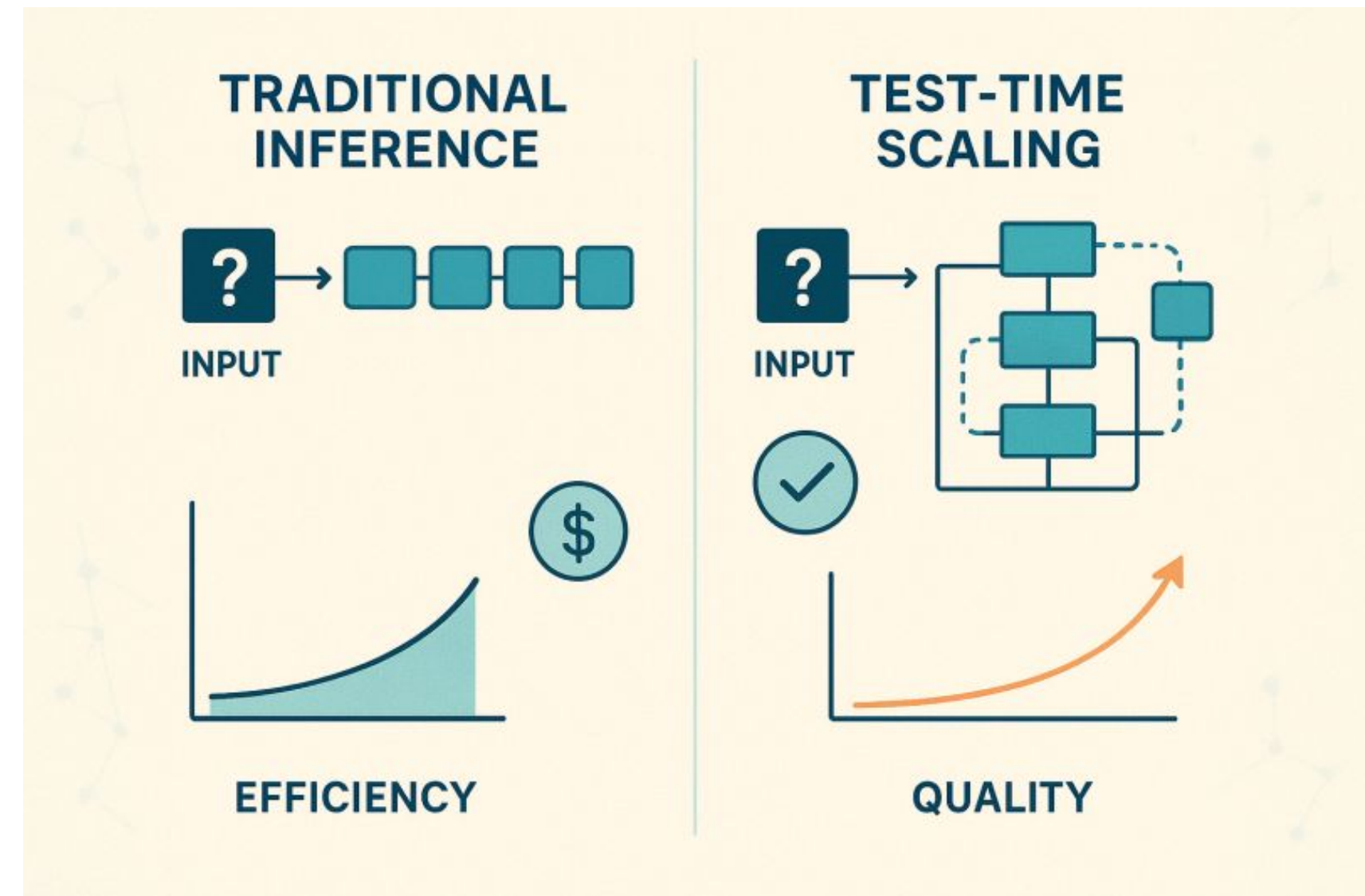
실무 활용 예)

중복된 경우 전략 합침, 규칙/전략 충돌시 최근 성공 우선 사용, 항목별 사용 빈도/성공 기여도 기록

TTS

- 1) 추론 시간 양상불 전략
추론 시에만 계산 시도 횟수를 n 회 늘려 더 좋은 답을 만드는 방식
답변을 여러번 생성하고 그 중 가장 낫다고 판단되는 답변을 선택하여 반환
Parallel TTS)
한 번에 여러개의 독립 샘플을 뽑아 hard voting
Sequential TTS)
실패 시에만 같은 문제를 재생성 및 반복
- 2) 한계
만들어 내는 샘플에 비례해 토큰/비용/지연 증가

Test-Time Scaling



MaTTS

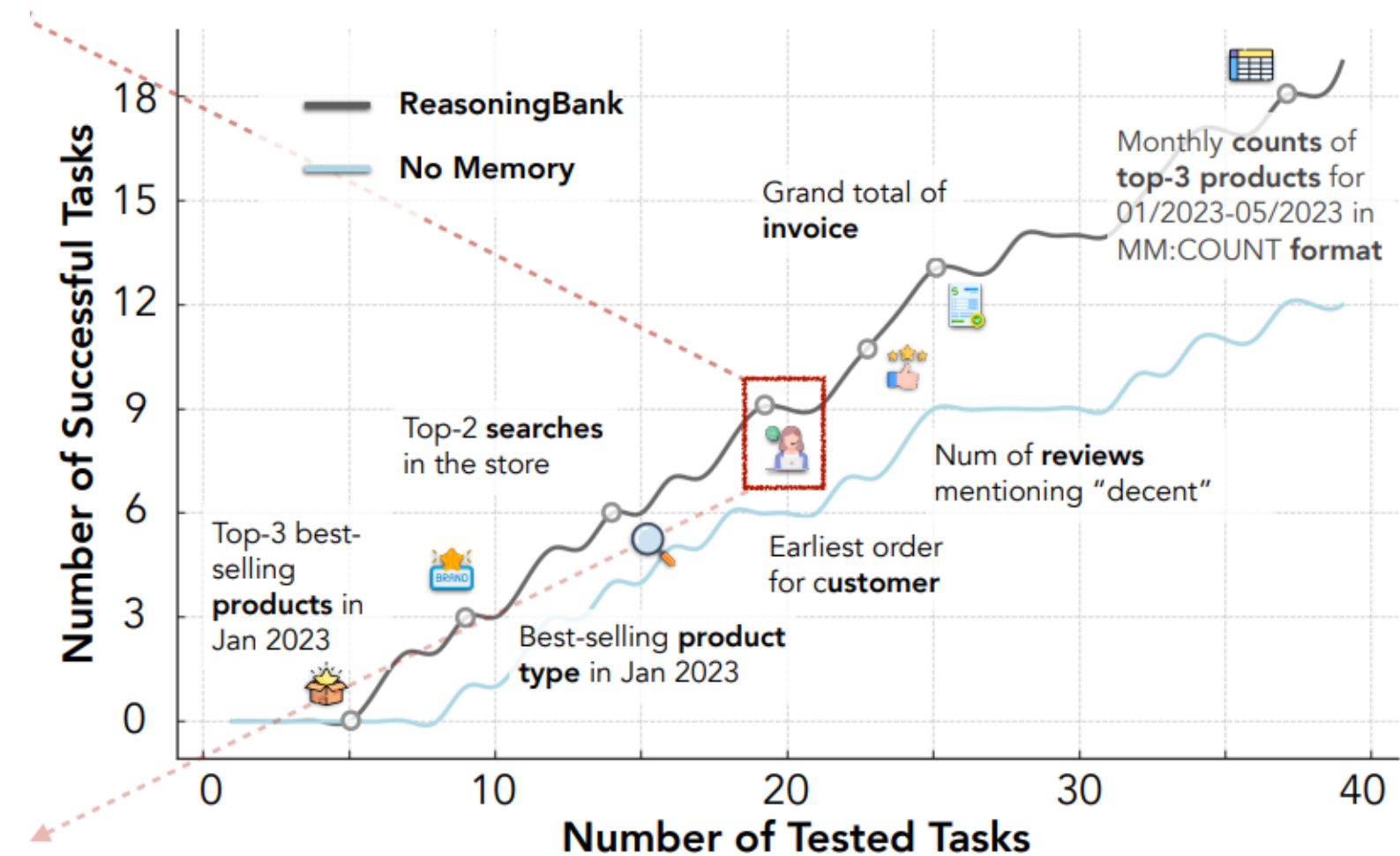
1) 기존 TTS의 문제점

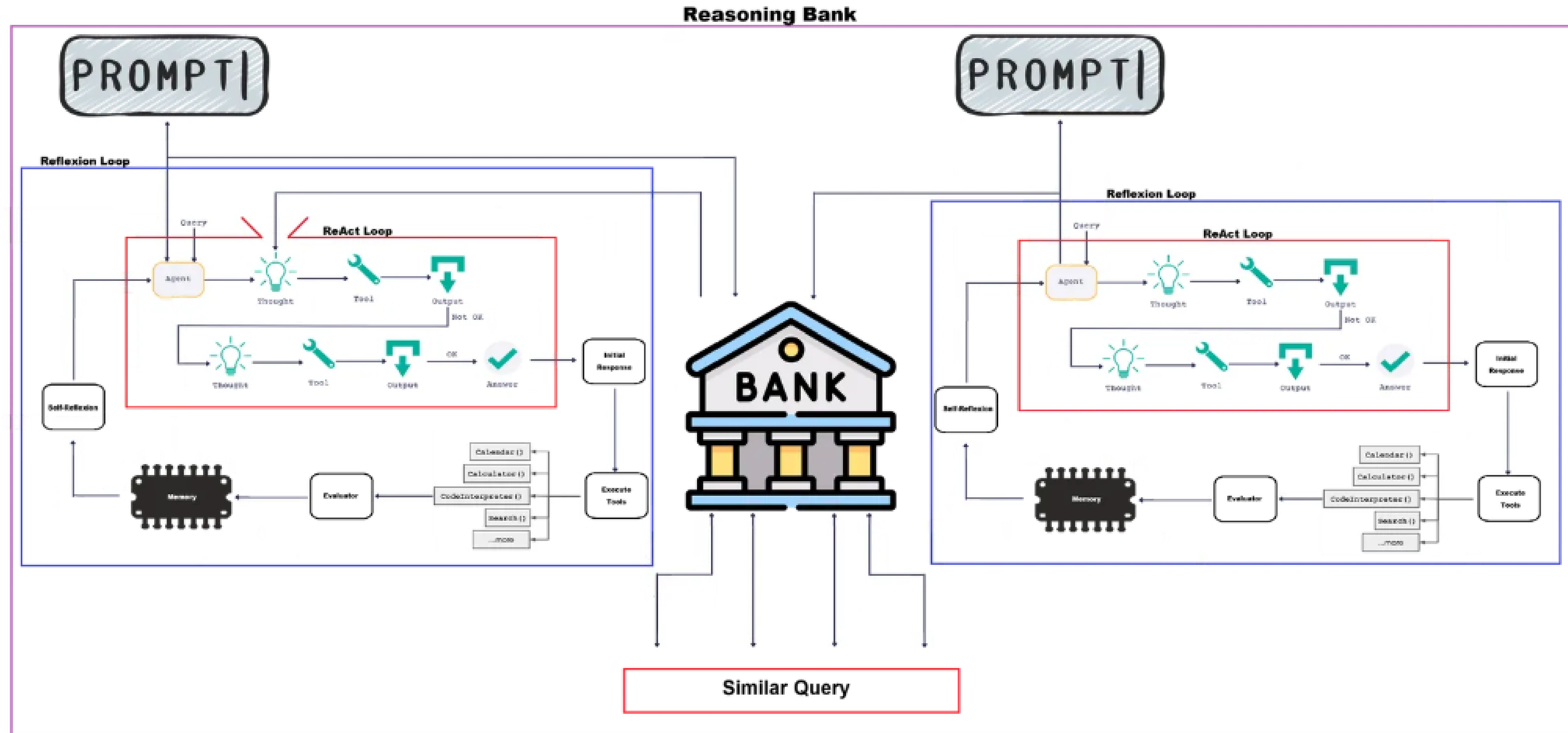
기존 TTS는 참조하는 메모리 없이 관련된 답변 샘플 n개를 무작위 생성하였기 때문에, 의미없는 토큰 사용이 발생함

2) 의미를 담고, 효율적인 답변 생성

memory를 통해 가장 효율적인 답변 샘플들을 여러 개 생성할 수 있기 때문에 지연/토큰 낭비를 막아냄
이전 Task를 참조할 수 있기 때문에 더욱 신뢰도 높은 답변을 생성함
여러 답변 중에 일관된 패턴의 답변만 남기고 모순되거나 신뢰도 떨어지는 답변을 빠르게 무시함

Memory-aware Test-Time Scaling





ReAct(Reasoning 부분)

```

class ReActAgent:
    """ReAct 프레임워크: Reasoning과 Acting 결합"""

    def __init__(self, reasoning_bank: ReasoningBank, reflexion: Reflexion):
        self.reasoning_bank = reasoning_bank
        self.reflexion = reflexion
        self.current_steps: List[ReActStep] = []

    def think(self, thought: str) -> str:
        """Reasoning 단계: 생각을 정리"""
        return thought

    def _generate_answer_from_history(self, self, task_description: str) -> str:
        """이전 스텝의 검색 결과를 기반으로 구체적인 답변 생성"""
        # 이전 검색 결과 추출
        search_results = []
        for step in self.current_steps:
            if step.action == "search" and "검색 결과:" in step.observation:
                # observation에서 데이터 추출
                observation = step.observation
                if "개의 관련 정보를 찾았습니다" in observation:
                    # 실제 데이터 파싱
                    lines = observation.split("\n")
                    for line in lines[1:]: # 첫 줄 제외
                        if ": {" in line:
                            search_results.append(line)

        # 최근 ReasoningBank 검색 결과 직접 가져오기
        bank_memories = self.reasoning_bank.get_memory_history()
        recent_searches = [m for m in bank_memories if m.get('retrieved_knowledge')]

```

```

        answer_parts = []

        # 질문 타입 분석
        task_lower = task_description.lower()

        for memory in recent_searches[-3:]: # 최근 3개 검색 결과만 사용
            for item in memory.get('retrieved_knowledge', []):
                data = item.get('data', {})
                source = item.get('source', '')

                # 질문에 맞는 정보 추출
                if "어디에서 태어났나요" in task_description or "born" in task_lower:
                    if source == 'bio' and 'born_in_city' in data:
                        answer_parts.append(f"{data.get('title', '')}은(는) {data.get('born_in_city', '')}에서 태어났습니다.")

                elif "생년" in task_description or "born_year" in task_lower or "언제" in task_description:
                    if source == 'bio' and 'born_year' in data:
                        answer_parts.append(f"{data.get('title', '')}의 생년은 {data.get('born_year', '')}년입니다.")

                elif "수도" in task_description or "capital" in task_lower:
                    if source == 'world' and 'capital' in data:
                        answer_parts.append(f"{data.get('title', '')}의 수도는 {data.get('capital', '')}입니다.")

                elif "어떤 나라" in task_description or "country" in task_lower:
                    if source == 'corpus' and 'country' in data:
                        answer_parts.append(f"{data.get('title', '')}은(는) {data.get('country', '')}에 있습니다.")

                elif "인구" in task_description or "population" in task_lower:
                    if source == 'corpus' and 'population_k' in data:
                        answer_parts.append(f"{data.get('title', '')}의 인구는 약 {data.get('population_k', '')}천 명입니다.")

                # 일반적인 경우: 모든 정보 통합
                if not answer_parts and data:
                    info_str = ", ".join([f"{k}: {v}" for k, v in data.items() if k != 'title'])
                    if info_str:
                        answer_parts.append(f"{data.get('title', '')}: {info_str}")

        if answer_parts:
            return "\n".join(answer_parts[:3]) # 최대 3개까지만
        else:
            return "검색 결과를 바탕으로 답변을 찾았지만, 구체적인 정보를 추출하지 못했습니다."

```

ReAct(Action 부분)

```
def act(self, action: str, action_input: Dict[str, Any]) -> str:
    """Acting 단계: 액션 실행"""
    if action == "search":
        query = action_input.get("query", "")
        results = self.reasoning_bank.search(query)
        if results:
            # 더 상세한 정보 제공
            detail_str = ""
            for r in results[:3]:
                data = r.get('data', {})
                source = r.get('source', '')
                if source == 'bio':
                    detail_str += f"\n[{source}] {data.get('title', '')}: {data.get('bio', '')}\n"
                    (출생지: {data.get('born_in_city', '')}, 출생년도: {data.get('born_year', '')})"
                elif source == 'corpus':
                    detail_str += f"\n[{source}] {data.get('title', '')}: {data.get('summary', '')}\n"
                    (국가: {data.get('country', '')}, 인구: {data.get('population_k', '')}천 명)"
                elif source == 'world':
                    detail_str += f"\n[{source}] {data.get('title', '')}: 수도 {data.get('capital', '')}, 통화 {data.get('currency', '')}"

            return f"검색 결과: {len(results)}개의 관련 정보를 찾았습니다.{detail_str}"
        else:
            return "관련 정보를 찾을 수 없습니다."
```

```
elif action == "update":
    key = action_input.get("key", "")
    field = action_input.get("field", "")
    value = action_input.get("value", "")
    source = action_input.get("source", "bio")
    self.reasoning_bank.update(key, field, value, source)
    return f"업데이트 완료: {key}.{field} = {value}"

elif action == "reflect":
    task = action_input.get("task", "")
    memories = self.reflexion.get_relevant_memories(task, top_k=2)
    if memories:
        return f"과거 경험 ({len(memories)}개):\n" + \
            "\n".join([f"- 에피소드 {m.episode_num}: {m.lesson_learned}"
                        for m in memories])
    else:
        return "관련 과거 경험이 없습니다."

elif action == "finish":
    user_answer = action_input.get("answer", "")
    task = action_input.get("task", "")

    # 사용자가 답변을 제공했으면 사용, 아니면 자동 생성
    if user_answer and "답변을 찾았습니다" not in user_answer:
        return f"작업 완료: {user_answer}"
    else:
        # 이전 스텝에서 검색 결과를 기반으로 답변 자동 생성
        generated_answer = self._generate_answer_from_history(task if task else "")
        if generated_answer:
            return f"답변: {generated_answer}"
        else:
            return f"작업 완료: {user_answer if user_answer else '답변을 찾았습니다.'}"

else:
    return f"알 수 없는 액션: {action}"
```


Reflexion

```
class Reflexion:
    """이전 에피소드에서 학습한 메모리를 저장하는 시스템"""

    def __init__(self):
        self.memories: List[ReflexionMemory] = []

    def add_memory(self, episode_num: int, task_description: str,
                  success: bool, failure_reason: Optional[str] = None,
                  lesson_learned: str = ""):
        """새로운 메모리 추가"""
        memory = ReflexionMemory(
            episode_num=episode_num,
            task_description=task_description,
            success=success,
            failure_reason=failure_reason,
            lesson_learned=lesson_learned,
            timestamp=datetime.now().isoformat()
        )
        self.memories.append(memory)
```

```
def get_relevant_memories(self, task_description: str, top_k: int = 3) -> List[ReflexionMemory]:
    """관련 메모리 검색 (간단한 키워드 매칭)"""
    task_lower = task_description.lower()
    scored_memories = []

    for memory in self.memories:
        score = 0
        memory_lower = memory.task_description.lower()

        # 키워드 매칭 점수
        task_words = set[str](task_lower.split())
        memory_words = set[str](memory_lower.split())
        common_words = task_words & memory_words
        score += len(common_words)

        # 실패 메모리에 더 높은 가중치
        if not memory.success:
            score += 2

        scored_memories.append((score, memory))

    scored_memories.sort(key=lambda x: x[0], reverse=True)
    return [mem for _, mem in scored_memories[:top_k]]
```

Reasoning Bank

```
class ReasoningBank:
    """지식 베이스를 저장하고 검색하는 시스템"""

    def __init__(self, bio_path: str, corpus_path: str, world_path: str):
        self.bio_path = bio_path
        self.corpus_path = corpus_path
        self.world_path = world_path

        # 데이터 로드
        self.bio_db = pd.read_csv(bio_path) if os.path.exists(bio_path) else pd.DataFrame()
        self.corpus_db = pd.read_csv(corpus_path) if os.path.exists(corpus_path) else pd.DataFrame()
        self.world_db = pd.read_csv(world_path) if os.path.exists(world_path) else pd.DataFrame()

        # 메모리 히스토리
        self.memory_history: List[BankMemory] = []

        print(f"[ReasoningBank] 로드된 데이터:")
        print(f"  - Bio: {len(self.bio_db)} 개")
        print(f"  - Corpus: {len(self.corpus_db)} 개")
        print(f"  - World: {len(self.world_db)} 개")
```

```
def search(self, query: str, context: Optional[str] = None) -> List[Dict[str, Any]]:
    """지식 베이스에서 검색"""
    results = []

    if not self.bio_db.empty:
        bio_matches = self.bio_db[
            self.bio_db['title'].str.contains(query, case=False, na=False) |
            self.bio_db['bio'].str.contains(query, case=False, na=False)
        ]
        for _, row in bio_matches.iterrows():
            results.append({
                'source': 'bio',
                'data': row.to_dict()
            })

    if not self.corpus_db.empty:
        corpus_matches = self.corpus_db[
            self.corpus_db['title'].str.contains(query, case=False, na=False) |
            self.corpus_db['summary'].str.contains(query, case=False, na=False)
        ]
        for _, row in corpus_matches.iterrows():
            results.append({
                'source': 'corpus',
                'data': row.to_dict()
            })

    if not self.world_db.empty:
        world_matches = self.world_db[
            self.world_db['title'].str.contains(query, case=False, na=False)
        ]
        for _, row in world_matches.iterrows():
            results.append({
                'source': 'world',
                'data': row.to_dict()
            })

    memory = BankMemory(
        query=query,
        retrieved_knowledge=results,
        updated_knowledge=None,
        timestamp=datetime.now().isoformat()
    )
    self.memory_history.append(memory)

    return results
```

```
def update(self, key: str, field: str, value: Any, source: str = 'bio'):
    """지식 베이스 업데이트"""
    updated_knowledge = {
        'key': key,
        'field': field,
        'value': value,
        'source': source
    }

    # 실제 데이터프레임 업데이트
    if source == 'bio' and not self.bio_db.empty:
        mask = self.bio_db['title'] == key
        if mask.any():
            self.bio_db.loc[mask, field] = value

    elif source == 'corpus' and not self.corpus_db.empty:
        mask = self.corpus_db['title'] == key
        if mask.any():
            self.corpus_db.loc[mask, field] = value

    elif source == 'world' and not self.world_db.empty:
        mask = self.world_db['title'] == key
        if mask.any():
            self.world_db.loc[mask, field] = value

    # 메모리에 저장
    memory = BankMemory(
        query=f"UPDATE: {key}.{field}",
        retrieved_knowledge=[],
        updated_knowledge=updated_knowledge,
        timestamp=datetime.now().isoformat()
    )
    self.memory_history.append(memory)
```

결과보고(Dataset: HotPotQA)

HotpotQA + OpenAI를 사용한 모든 조합 비교				
생성 일시: 2025-10-30 02:34:43				
종합 비교표				
조합	평균 성공률	평균 스텝 수	평균 검색 횟수	평균 Reflection 횟수
ReAct만	20.0%	2.0	0.0	0.0
Reflexion만	0.0%	1.0	0.0	2.4
ReasoningBank만	100.0%	1.0	3.4	0.0
ReAct + Reflexion	40.0%	2.0	0.0	2.0
ReAct + ReasoningBank	100.0%	3.0	1.0	0.0
Reflexion + ReasoningBank	80.0%	2.0	3.4	2.0
ReAct + Reflexion + ReasoningBank	100.0%	3.0	1.0	2.0

질문별 상세 비교					
질문 1: Were Scott Derrickson and Ed Wood of the same nationality?...					
조합	성공	스텝 수	검색	Reflection	답변 요약
ReAct만	✗	2	0	0	답변: Scott Derrickson and Ed Wood both being Americ...
Reflection만	✗	1	0	1	Scott Derrickson and Ed Wood were not of the same ...
ReasoningBank만	✓	1	1	0	Yes, Scott Derrickson and Ed Wood were of the same...
ReAct + Reflection	✓	2	0	2	답변: 네, Scott Derrickson과 Ed Wood는 동일한 국적인 미국인입니다. ...
ReAct + ReasoningBank	✓	3	1	0	답변: 예, Scott Derrickson과 Ed Wood는 같은 국적이었습니다.
Reflection + ReasoningBank	✗	2	1	2	검색 결과에 따르면, Scott Derrickson과 Ed Wood는 동일한 국적이었습니다...
ReAct + Reflection + ReasoningBank	✓	3	1	2	답변: 네, Scott Derrickson과 Ed Wood는 동일한 국적을 가지고 있습니다...
질문 2: What government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tel...					
조합	성공	스텝 수	검색	Reflection	답변 요약
ReAct만	✗	2	0	0	답변: 주어진 질문에 대한 답변을 찾기 위해 "Kiss and Tell" 영화에서 Corl...
Reflection만	✗	1	0	2	주어진 질문은 영화 'Kiss and Tell'에서 Corliss Archer 역할을 한 ...
ReasoningBank만	✓	1	3	0	The government position held by the woman who port...
ReAct + Reflection	✗	2	0	2	답변: 1. 먼저, 영화 "Kiss and Tell"에 나온 여성 주연 배우의 이름을 찾아...
ReAct + ReasoningBank	✓	3	1	0	답변: The government position held by the woman who ...
Reflection + ReasoningBank	✓	2	3	2	The government position held by the woman who port...
ReAct + Reflection + ReasoningBank	✓	3	1	2	답변: The woman who portrayed Corliss Archer in the ...

질문 3: What science fantasy young adult series, told in first person, has a set of companion books narratin...					
조합	성공	스텝 수	검색	Reflection	답변 요약
ReAct만	✗	2	0	0	답변: 주어진 정보에 따르면, 과학 판타지 장르의 청소년을 대상으로 한 첫 인칭으로 이야기...
Reflection만	✗	1	0	3	주어진 정보로는 정확히 어떤 시리즈인지 알기 어려운 것 같습니다. 더 자세한 정보가 필요할...
ReasoningBank만	✓	1	3	0	Animorphs
ReAct + Reflection	✗	2	0	2	답변: 주어진 정보에 따르면, 사용자가 찾는 책은 과학 판타지 장르의 청소년 시리즈이며, ...
ReAct + ReasoningBank	✓	3	1	0	답변: The science fantasy young adult series, told i...
Reflection + ReasoningBank	✓	2	3	2	주어진 정보에 따르면, 해당 소설은 "Animorphs"이라고 추정됩니다.
ReAct + Reflection + ReasoningBank	✓	3	1	2	답변: The answer is "Animorphs."
질문 4: Are the Laleli Mosque and Esma Sultan Mansion located in the same neighborhood?...					
조합	성공	스텝 수	검색	Reflection	답변 요약
ReAct만	✓	2	0	0	답변: No, the Laleli Mosque and Esma Sultan Mansion ...
Reflection만	✗	1	0	3	주어진 컨텍스트만으로는 Laleli Mosque와 Esma Sultan Mansion이 같...
ReasoningBank만	✓	1	5	0	아니요.
ReAct + Reflection	✓	2	0	2	답변: Given the inference that the two buildings are...
ReAct + ReasoningBank	✓	3	1	0	답변: Based on the search results provided, it was c...
Reflection + ReasoningBank	✓	2	5	2	컨텍스트를 고려하면, Laleli Mosque와 Esma Sultan Mansion이 동일...
ReAct + Reflection + ReasoningBank	✓	3	1	2	답변: No, according to the search results, the Lalel...

결과보고 3

질문 5: The director of the romantic comedy "Big Stone Gap" is based in what New York city?....

조합	성공	스텝 수	검색	Reflection	답변 요약
ReAct만	✗	2	0	0	답변: 1. "Big Stone Gap"의 감독을 확인해야 합니다.
Reflection만	✗	1	0	3	질문은 영화 "Big Stone Gap"의 로맨틱 코미디 감독이 뉴욕 시 어디에 기반을 두...
ReasoningBank만	✓	1	5	0	The director of the romantic comedy "Big Stone Gap...
ReAct + Reflection	✗	2	0	2	답변: 1. 먼저, 영화 "Big Stone Gap"의 감독을 확인해야 합니다.
ReAct + ReasoningBank	✓	3	1	0	답변: The director of the romantic comedy "Big Stone...
Reflection + ReasoningBank	✓	2	5	2	The director of the romantic comedy "Big Stone Gap...
ReAct + Reflection + ReasoningBank	✓	3	1	2	답변: The director of the romantic comedy "Big Stone...

<ReAct 2023/03/10>

ReAct: Synergizing Reasoning and Acting in Language Models - Shunyu Yao et al.

<Reflexion 2023/10/10>

Reflexion: Language Agents with Verbal Reinforcement Learning - Noah Shinn et al.

<Reasoning Bank 2025/09/29>

ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory - Siru Ouyang et al.

Thank You

ReAct / Reflexion / Reasoning Bank Paper Review

감사합니다.

QnA